Algorithms

hash tables, heaps, binary trees, linked lists, depth-first search, recursion, merge

- Divide and conquer Algorithm: works by recursively breaking down a problem into two or more sub problems of the same (or related type) until these become simple enough to be solved directly. The solution to sub-problems are then combined to give a solution to the original problem.
    - Every algorithm that uses recursion or loops could be regarded as a divide-and-conquer algorithm.
        - quick sort, merge sort,
        - binary search
        - multiplying large numbers, Karatsuba
        - syntactic analysis, top-down parsers
        - discrete Fourier transform
    - Divide and conquer typically splits the problem in half, solves each half, then stitches the halves back together to form a full solution.
- Tail recursion or tail-end recursion is a special case of recursion in which last operation performed in a recursive call.
    - Return a (usually simple) value without recursion.
- Dynamic programming typically removes one element from the problem, solves the smaller problem, and then uses the solution to this smaller problem to add back the element in the proper way.
    - Dynamic programming is probably the easiest algorithm design technique to apply in practice.
- Greedy algorithms, which make the best local decision at each step

---

Analysis of Algorithms

Types of Analyses

- Best case - Running time determined by easiest input
- Worst case - Running time guarantee for all inputs
- Average case - Running time for random input

Commonly-used notations

- Tilde - provide approximate model
- Big Theta - classify algorithms
- Big Oh - develop upper bounds
- Big Omega - develop lower bounds

Map Reduce

Big-O Notation:

1. Big-O notation provides an upper bound on the growth rate of the function.
2. Describes an algorithm's usage of computational resources.
3. Expressed as a function of the length of its input using
4. O(N) describes an algorithm whose performance will linearly and in direct proportion to the size of the input data set.
5. O(N^2) represents an algorithm whose performance is directly proportional to the square of the size of the input data set.
6. If we have any input of size N, solving it will require at most f(N) steps.

- Bubble Sort O(n^2)
- Selection sort O(n^2)
- Quick Sort O(n^2)
- Heap Sort O(N log N)

- Big Theta is more tightly bound than Big O. mean there exists two constants c1 and c2 such that c1n log n < f(n) < c2 log n
- Big Omega saya that the algorithen has a lower bound of c n log n
- Big O is not concerened with factors that do not chagne as the input increases.

- O(g) - asymptotic upper bound -
- omage **Ω** (g) - lower limit -
- theta $\theta(g) = O(g) \cap \Omega(g)$ - upper and lower bounds of a function are on the same order of magnitude.
    - A funciton is in big-theta of f if its not much worse but also not much better than f
- There is no asymptotic notation to describe the average case.
- dont congfure best / average /worst case with asymptotic bounds.

Issues:

1. Too hard to analyze
2. Average case unknown
3. Unknown constant: Big-O analysis only tells you how it grows with the size of the problem, not now efficient it is. e.g. Both walking and travelling at the speed of light have a time-as-function-of-distance of complexity O(N). Altho they have the same big-o characteristics, one is faster than other.
4. Small data set

Estimating for recursive and nested loops

- estimate the maximum number of times each loop can be executed.
- add these bounds for cycles following each other.
- multiply theses founds for nested cycles/parts of code.
- Time complexity of a while cycle is O(n).

References:

- http://www.topcoder.com/tc?module=Static&d1=tutorials&d2=complexity1

Sorting

Approaches to Sorting

- Selection of Data Structures
- Incremental Insertion Technique
  - a particularly useful technique in geometric algorithms
- Divide and Conquer Technique
  - Mergesort is a classic divide-and-conquer algorithm
- Randomization Technique
  - In quicksort, we separate the n-1 other items into two piles: a low pile containing all the elements that will appear before p in sorted order and a high pile containing all the elements that will appear after p in sorted order. After sorting the two piles and arranging the piles correctly, we have succeeded in sorting the n items.
- Bucketing Techniques
  - Bucketing is a very effective idea whenever we are confident that the distribution of data will be roughly uniform. It is the idea that underlies hash tables, kd-trees, and a variety of other practical data structures. The downside of such techniques is that the performance can be terrible whenever the data distribution is not what we expected.

Sorting methods:

- Insertion, selection, merge, exchange, partitioning
- in other words: Swap based, Merge based, Tree based
- cost of sorting - comparisons and exchanges

Stability:

- Stable sorting algorithms maintains the relative order of records with equal keys.
  - one way of doing this to artificially extend the key comparision, so that comparisons between two objects with othwerwise equal keys are decided using the order of the entries in the original data order as a tie-breaker.
  - (4, 2) (3, 7) (3, 1) (5, 6) can be sorted in two way
  - (3, 7)..(3, 1).....
  - (3, 1)..(3, 7).....

v • d • e	
Theory	Computational complexity theory • Big O notation • Total order • Lists • Stability • Comparison sort
Exchange sorts	Bubble sort • Cocktail sort • Odd-even sort • Comb sort • Gnome sort • **Quicksort**
Selection sorts	Selection sort • Heapsort • Smoothsort • Cartesian tree sort • Tournament sort
Insertion sorts	Insertion sort • Shell sort • Tree sort • Library sort • Patience sorting
Merge sorts	Merge sort • Polyphase merge sort • Strand sort • Timsort
Non-comparison sorts	Radix sort • **Bucket sort** • Counting sort • Pigeonhole sort • Burstsort • Bead sort
Others	Topological sorting • Sorting network • Bitonic sorter • Batcher odd-even mergesort • Pancake sorting
Ineffective/humorous sorts	Bogosort • Stooge sort

	Time					
Sort	Average	Best	Worst	Space	Stability	Remarks
Bubble sort	O(n^2)	O(n^2)	O(n^2)	Constant	Stable	Always use a modified bubble sort
Modified Bubble sort	O(n^2)	O(n)	O(n^2)	Constant	Stable	Stops after reaching a sorted array
Selection Sort	O(n^2)	O(n^2)	O(n^2)	Constant	Stable	Even a perfectly sorted input requires scanning the entire array
Insertion Sort	O(n^2)	O(n)	O(n^2)	Constant	Stable	In the best case (already sorted), every insert requires constant time
Heap Sort	O(n*log (n))	O(n*log (n))	O(n*log(n))	Constant	Instable	By using input array as storage for the heap, it is possible to achieve constant space
Merge Sort	O(n*log (n))	O(n*log (n))	O(n*log(n))	Depends	Stable	On arrays, merge sort requires O(n) space; on linked lists, merge sort requires constant space
Quicksort	O(n*log (n))	O(n*log (n))	O(n^2)	Constant	Stable	Randomly picking a pivot value (or shuffling the array prior to sorting) can help avoid worst case scenarios such as a perfectly sorted array.

- Heapsort is nothing but an implementation of selection sort with the right data structure.
-

Table 2 Strengths and Weaknesses

Sort	Order	Worst Case	Memory	Stable	DataTypes	Complexity
MSD Radix	N	N	Nk + Np + R + stack	yes	strings	hi
Ternary Quick	N log N	??	Nk + Np + stack	yes	strings	hi
Quick	N log N	N^2	Nk + Np + stack	no	all	hi
Merge	N log N	N log N	Nk + 2Np + stack	yes	all	medium
Heap	N log N	N log N	Nk + Np	no	all	medium
Comb	N log N	??	Nk + Np	no	all	low
Shell	N (log N)^2	??	Nk + Np	no	all	low
Insertion	N^2	N^2	Nk + Np	yes	all	very low
Selection	N^2	N^2	Nk + Np	yes	all	very low

Quick Sort

- Worst O(n^2), Best O(n Log n), Average O(n log b)

1. Pickup an element from the list, called a **pivot,**
2. Reorder the list so that
    a. all elements with value less than the pivot come before the pivot
    b. all elements with values greater than the pivot come after the pivot
3. Recursively sort the sub-list of lesser elements and the sub-list of greater elements.

- With in-place partitioning uses only constant additional space before making any recursive all.
- variations: **Balanced quicksort**, **esternal quicksort**, **three-way radix quicksort**
- Quicksort is a space-optimized version of the binary tree sort. Instead of inserting items sequentially into an explicit tree, quick sort organized them concurrently into a tree that is implied by the recursive calls.
- Competes with merge sort and heap sort.
- **Bucket sort** with two buckets is very similar to quicksort; the pivot in this case is effectively the value in the middle of the value range,

References:

- http://www.algolist.net/Algorithms/Sorting/Quicksort
- http://en.wikibooks.org/wiki/Algorithm_Implementation/Sorting/Quicksort#Java

Bubble Sort

- Compare each pair of adjacent elements from the beginning of an array and, if they are in reversed order, swap them.
- if at least one swap has been done, repeat step 1.

Selection Sort

- Array is imaginary divided into two parts - sorted one and unsorted one
- At the beginning, sorted part is empty, while unsorted one contains whole array.
- At every step algorithm finds minimal element in the unsorted part and adds it to the end of the sorted one.
- When unsorted part becomes empty, algorithm stops.
- n+ (n-1) + (n-2)... = 1 results in O(n^2)

Insertion Sorts

- Insertion sort somewhat resembles **selection sort**.
- Array is imaginary divided into two parts.
- At the beginning, sorted part contains first element of the array and unsorted one contains the rest.
- At every step, algorithm takes first element in the unsorted part and inserts it to the right place of the sorted one.
- Variation of this sort is called **binary insertion sort**.
- Although insertion sort takes in the worst case, it will perform considerably better if the data is almost sorted.
- Efficient for small list
- O(N^2)

Shell sort

- Improves bubble sort and insertion sort by moving out of order elements more than one position at a time.

Merge Sort

- merges already sorted lists into new sorted list.
- Scales well to very large lists
- wors-case running time is O(n log n)
- is being used in java since 1.3
- JDK7 timsort
- Divides the unsorted list into two sublists of about half the size.
- Sort each sublist recursively by re-applying merge sort.
- Merge the two sublists back into one sorted list.
- TBD

Heap Sort

- an efficient version of **selection sort.**
- uses heap data structure - a special type of binary tree.
- Once the data list has been made into a heap, the root node is guaranteed to be the largest (or smallest) element.
- O(n log n)
- min or max heap.
- TBD

Radix Sort

- sorts numbers by processing individual digits

Search

Binary Search

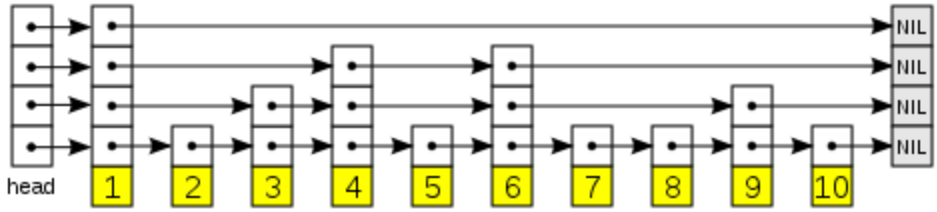
- Get the middle element
- if the middle element equals to the searched value, the algorithjem stops
- otherwise
    - searched value is less, than the middle element,
    - searched value is greater than the middle element.

```
int binarySearch(int data[], int value, int leftIndex, int rightIndex)
{
     while (leftIndex <= rightIndex)
     {
          int middle = (leftIndex + rightIndex) /2;

          if(data[middle]==value)
               return middle;
          else if (data[middle] > value)
               right = middle-1;
          else
               left = middle+1
     }
     return -1;
}
```

Data Structure

- **Unsorted linked lists** or arrays - For small data sets, say up to 10 to 20 items,
- **Self-organizing list-** Whenever a key is accessed or inserted, always move it to head of the list. Self-organizing data structures can be built from arrays as well as linked lists.
- **Sorted linked lists** or arrays - A sorted array will be appropriate if and only if there are not many insertions or deletions.
- **Hash tables** - For applications involving a moderate-to-large number of keys (say between 100 and 1,000,000), a hash table with bucketing is probably the right way to go. In a hash table, we use a function that maps keys (be they strings, numbers, or whatever) to integers between 0 and m-1. We maintain an array of m buckets, each typically implemented using an unsorted linked list. For a given key, the hash function immediately identifies which bucket will contain it. If we use a hash function that spreads the keys out nicely and a sufficiently large hash table, each bucket should contain very few items, thus making linear search acceptable. Insertion and deletion from a hash table reduce to insertion and deletion from the bucket/list.
    - A well-tuned hash table will likely outperform a sorted array in most applications. However, several design decisions go into creating a well-tuned hash table:
    - How big should the table be? Typically, m should about the same as the maximum number of items you expect to put in the table. Make sure that m is a prime number, so as to minimize the dangers of a bad hash function.
    - What hash function should I use? For strings, something like
- **Binary search trees** - Binary search trees are elegant data structures that support fast insertions, deletions, and queries. IIn random search trees, no rebalancing takes place and we simply insert a node at the leaf position where we can find it. Although search trees perform well under random insertions, most applications are not really random. Indeed, unbalanced search trees constructed by inserting keys in sorted order are a disaster, performing like a linked list.
- **Balanced search trees -** use local rotation operations to restructure search trees, moving more distant nodes closer to the root while maintaining the in-order search structure of the tree.
    - Among balanced search trees, **AVL** and **2/3 trees**. **Red-black trees** seem to be more popular.
    - A particularly interesting self-organizing data structure is the **splay tree**, which uses rotations to move any accessed key to the root. Frequently used or recently accessed nodes thus sit near the top of the tree, allowing fast search.
- **B-trees** - For data sets so large that they will not fit in main memory (say more than 1,000,000 items) your best bet will be some flavor of a B-tree.The idea behind a B-tree is to collapse several levels of a binary search tree into a single large node, so that we can make the equivalent of several search steps before another disk access is needed.
- **Skip lists** - These are somewhat of a cult data structure. Their primary benefits seem to be ease of implementation relative to balanced trees.



Dictionary

Input description: A set of n records, each identified by one or more key fields.
Problem description: Build and maintain a data structure to efficiently locate, insert, or delete the record associated with any query key q.
Discussion: Dozens of different data structures have been proposed for implementing dictionaries including
hash tables,
skip lists, and
balanced/unbalanced binary search trees

An essential piece of advice is to carefully isolate the implementation of the dictionary data structure from its interface. Use explicit calls to subroutines that initialize, search, and modify the data structure, rather than embedding them within the code. This leads to a much cleaner program, but it also makes it easy to try different dictionary implementations to see how they impact performance. Do not obsess about the cost of the procedure call overhead inherent in such an abstraction. If your application is so time-critical that such overhead can impact performance, then it is even more essential that you be able to easily experiment with different implementations of your dictionary.

In choosing the right data structure for your dictionary, ask yourself the following questions:

- How many items will you typically have in your data structure?
- Do you know the relative number of insertions, deletions, and search queries?
- Will there be any modifications to the data structure after it is first constructed?
- Relative frequency with which different keys will be accessed?
- Is it critical that individual operations be fast, or only that the total amount of work done over the entire program be minimized .

Reference:

- Algorithm Design Manual

Hash Function

- Hash Function is any well-defined procedure or mathematical function that converts a large, possibly variable-sized amount of data into a small datum, usually a single integer that may serve as an index to an array.
- Hash code collision.

Data Distribution:

This is the measure of how well the hash function distributes the has values of elements with-in a set of data.

Hash Function Efficiency:

This is the measure of how efficiently the hash function produces has values for elements within a set of data. Generally assumed that has functions have a complexity of O(1).

Hashing Methodologies

There are two main methodologies for a hash algorithm

- Additive and Multiplicative Hashing e.g. RS Hash Function
- Rotative Hashing e.g. JS Hash Function

```
public long RSHash(String str)
 {
    int b     = 378551;
    int a     = 63689;
    long hash = 0;

    for(int i = 0; i < str.length(); i++)
    {
       hash = hash * a + str.charAt(i);
       a    = a * b;
    }

    return hash;
 }
 /* End Of RS Hash Function */


 public long JSHash(String str)
 {
    long hash = 1315423911;

    for(int i = 0; i < str.length(); i++)
    {
       hash ^= ((hash << 5) + str.charAt(i) + (hash >> 2));
    }

    return hash;
 }
 /* End Of JS Hash Function */
 //^Bitwise exclusive OR
```

Prime Number vs pseudo random number generators: Best results have been found with the use of prime numbers.

Types of Hash Functions

- Perfect hashing
- Heuristic hashing/Special purpose hash functions
- Minimal perfect hashing
- Hashing data distribution / uniform distribution
- General purpose hash functions / special purpose hash functions

- Universal hashing

References:

- http://en.wikipedia.org/wiki/Hash_function
- http://www.partow.net/programming/hashfunctions/index.html

HashTable

- A data structure that uses a hash function to map identifying values, known as key, to their associated values.
- *Hash function* is used to transform the key into the index (the hash) of an *array of element* (slot or bucket) where the corresponding value is stored.
- Most hash table design accommodates hash collisions
- A well-dimensioned hash table, the average cost for each look-up is independent of the number of elements stored in the table.
- Most hash table designs allow arbitrary insertions and deletions of key-value pairs, at constant average cost per operation.
- In many situations, hash tables turn out to be more efficient than search trees or any other table lookup.
- Hash table is a simple array of items. Hash number is used to calculate an index from the key and used to place the data into the array.
- Hash function should provide a uniform distribution of hash values. A non-uniform distribution increases the number of collisions, and cost of resolving them.
- Perfect hash function ... no collisions.

Dynamic Resizing:

- Rehash us used to increase the capacity of and internally reorganizes hashtable, in order to accommodate and access its entries more efficiently.
- Java hashMap .. default factor 0.75, buckets are implemented using dynamic array (ArrayList). Dynamix Arrays are not linked list. They start with an initial size and then grow (usually twice the initial size) when needed. In memory data copy happens from the old and new ... expanded array.

Collision Resolutions:

Load Factor:

- The load of a hash table is the number of object in the hash table divided by the size of the table.

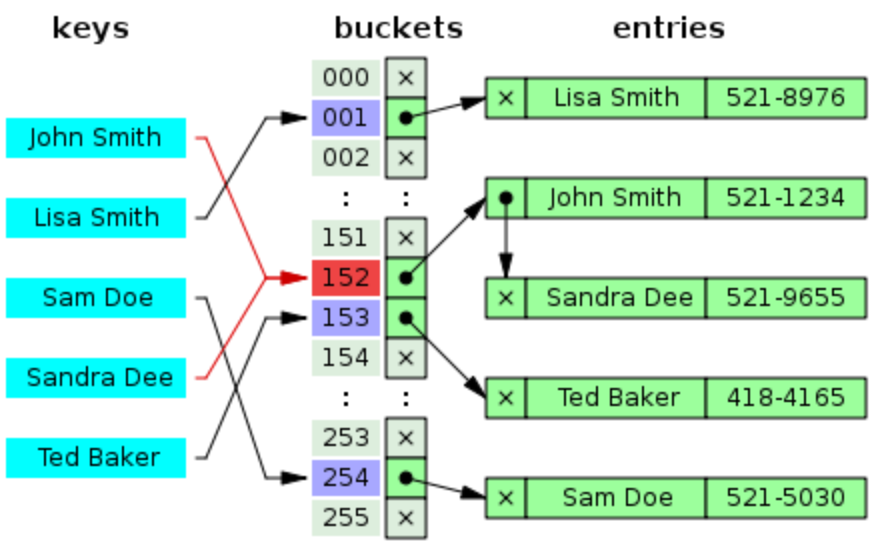
- With a good hash function the average lookup cost is nearly constant as load factor increases from 0 to 0.7. Beyond this point, the probability of collisions and the cost of handling them increases. Under 3/4
- Performance of collision resolution methods depends on this factor.
- On the other hand as the load factor approaches zero, the size of the hash table increases with little improvement in the search cost, and memory is wasted.

Separate chaining:

- also know as direct chaining or simply chaining each slot of the bucket array is pointer to a linked list that contains the key-value pairs that hashed to the same location. Lookup requires scanning the list for an entry with the given key. Insertion requires adding a new entry record to either end of the list.
- Chaining with list head.
- Chaining with other structures.

Open addressing / closed hashing

- all entry records are stored in the bucket array itself.
- When a new entry has to be inserted, the buckets are examined, starting with the hashed-to slot and proceeding in some probe sequence.
- Well-known probe sequence include: linear probing, quadratic probing, double hashing.



Counting sort, radix sort, BSTs to Quick sort, Internval trees, Skip Lists, Table Doubling, Self-organizing Lists

---

Trees and Graphs

Binary Tree

- Mathematically, it is an ordered directed graph, more specifically an arborescence: an acyclic connected graph where each node has zero or more children nodes and at most one parent node.
- a tree data structure in which each node has at most two child nodes.
- Nodes with children are parent nodes
- Root node the ancestor of all nodes. has no parent.
- Any node can be reached by starting at root node and repeatedly following references to either the left or right child.
- Leaf node has no children.
- Depth of a node n is the length of the path from the root to the node.
- height of a tree is the length of the path from the root to the deepest node in the tree.
- Siblings are nodes that share the same parent node.

Tree Traversal

- pre-order - root node is visited before its sub tree.
- post-order if root node is visited after
- in-order - traversal visits the nodes in increasing order.
- Depth-first order- visit the node farthest from the root that we can, but it must be child of a node we have already visited.
- Breath-first order- visit the nodes closet to the root that it has no already visited.

Binary search tree

- left subtree of a node contains only nodes with keys less than the node's key
- right subtree of a node contains only nodes with the keys greater than the node's key.
- Both the left and right subtrees must also be a binary search trees.
- No Duplication

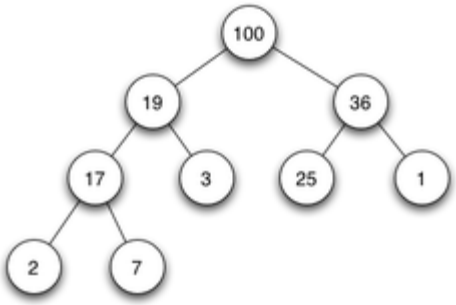
Deletion cases

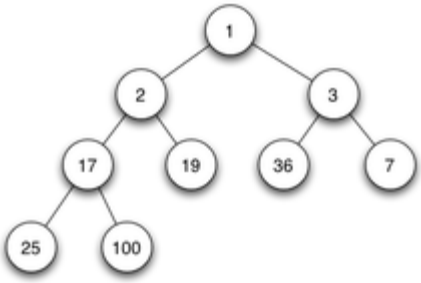
- Node with no children
- Node with one children
- Node with two children -> replace it with the smallest from the right.

Quad Tree

Binary Heap

- Binary tree with two additional constraints
    - The shape property: the tree is a complete binary tree, that is all levels of the tree except possibly the last one are fully filed.
    - each node is greater than or equal to each of its children.





B-Tree

- K = order of the tree. **B**ranching **O**ccupancy **B**alance **O**rdering
- Occupancy
    - All nodes (except the root) have at least “K” and at most “2K” data items.
    - the root has between 1 and 2K.
- Branching
    - Non-terminal node with M items has M+1 children,
    - Because of occupancy property, M is always between K and 2K - except at the root.
- Ordering
    - The key values in the data items in all nodes are in increasing order.
- Balance
    - All terminal nodes are on the same level, D, which is the depth of the tree.

Example:

- B-Tree of order 2
    - Root has at least 1 and at most 4 data items
    - Node has at least 2 and at most 4 data items

Insertion

- First search the tree to see if X already exists.
- It it does not, it needs to be added while preserving the properties of the B-Tree.
- Worst case is still O(log N) operation since at most one node is split at each level of the tree.

Deleting an Element

- Start by searching for the key value to be removed
- Like insertion, start at the bottom and works up the tree
- Must preserve the properties of the B-Tree.
- If node has only one item, then there are typically two cases:
    - An adjacent sibling of the empty node has a spare item.
    - There are no adjacent siblings with a spare item.

Reference:

- http://www.macs.hw.ac.uk/flex/BScCS/ds1/
- http://cslibrary.stanford.edu/110/BinaryTrees.html
- http://people.ksp.sk/~kuko/bak/index.html
- http://users.cis.fiu.edu/~weiss/dsaajava/code/DataStructures/

Suffix Trees and Arrays / Trie

Input description: A reference string S.
Problem description: Build a data structure for quickly finding all places where an arbitrary query string q is a substring of S.

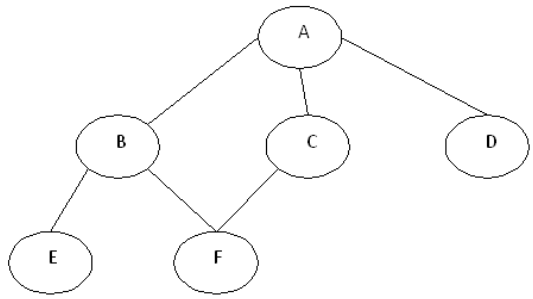
Minimum Spanning Tree

The minimum spanning tree (MST) of a graph defines the cheapest subset of edges that keeps the graph in one connected component. Telephone companies are particularly interested in minimum spanning trees, because the minimum spanning tree of a set of sites defines the wiring scheme that connects the sites using as little wire as possible. It is the mother of all network design problems.

- System.out.print(localRoot.data + " ");
- preOrder(localRoot.left);
- preOrder(localRoot.right);

- inOrder(localRoot.left);
- System.out.print(localRoot.data + " ");
- inOrder(localRoot.right);

- postOrder(localRoot.left);
- postOrder(localRoot.right);
- System.out.print(localRoot.data + " ");



Depth First Search

- DFS is an uninformed search that progresses by expanding the first child node of the search tree that appears and thus going deeper and deeper until a goal node is found, or until it hits a node that has no children.
- Start at vertex v, visit its neighbour w, then w's neighbour y and keep going until reach

'a dead end' then iterate back and visit nodes reachable from second last visited vertex and keep applying the same principle.

- always attempt to visit the node farthest from the root
- Stack is used in the implementation of the depth first search

Step 1: Push the root node in the Stack.
Step 2: Loop until stack is empty.
Step 3: Peek the node of the stack.
Step 4: If the node has unvisited child nodes, get the unvisited child node, mark it as traversed and push it on stack.
Step 5: If the node does not have any unvisited child nodes, pop the node from the stack.
Output - "A B E F C D"

```
dfs(node start) {
 stack s;
 s.push(start);
 while (s.empty() == false) {
  top = s.top();
  s.pop();
  mark top as visited;

  check for termination condition

  add all of top's unvisited neighbors to the stack.
  mark top as not visited;
 }
}
```

ANOTHER

```
dfs(node current)
{
 mark current as visited;
 visit all of current's unvisited neighbors by calling dfs(neighbor)
 mark current as not visited;
}
```

ANOTHER

```
dfs(vertex v) {
   visit(v);
   for each neighbor w of v
       if w is unvisited{
       dfs(w);
       add edge vw to tree T  } }
```

Breadth First Search (BFS)

- traverse the graph as close as possible to the root node.
- Queue is used in the implementation of the breadth first search.

Step 1: Push the root node in the Queue.
Step 2: Loop until the queue is empty.
Step 3: Remove the node from the Queue.
Step 4: If the removed node has unvisited child nodes, mark them as visited and insert the unvisited children in the queue.
output. "A B C D E F".

```
void bfs(node start) {
 queue s;
 s.push(start);
 while (s.empty() == false) {
  top = s.front();
  s.pop();
  mark top as visited;
check for termination condition (have we reached the node we want to?) add all of top's
unvisited neighbors to the stack.
 }
}
```

```
   bfs(G)
    {
    list L = empty
    tree T = empty
    choose a starting vertex x
    search(x)
    while(L nonempty)
        remove edge (v,w) from start of L
        if w not yet visited
        {
        add (v,w) to T
        search(w)
        }
    }

    dfs(G)
    {
    list L = empty
    tree T = empty
    choose a starting vertex x
    search(x)
    while(L nonempty)
        remove edge (v,w) from end of L
```

```
        if w not yet visited
        {
        add (v,w) to T
        search(w)
        }
    }

    search(vertex v)
    {
    visit(v);
    for each edge (v,w)
        add edge (v,w) to end of L
    }
```

Reference: http://www.ics.uci.edu/~eppstein/161/960215.html

Comparison

- advantage of DFS is that it has much lower memory requirements than BFS, because it's not necessary to store all of the child pointers at each level.
- given a family tree if one were looking for someone on the tree who's still alive,
    - person would be on the bottom of the tree.
    - BFS would take a very long time to reach that last level.
    - DFS, would find faster.
- if one were looking for a family member who died a very long time ago,
    - person would be closer to the top of the tree.
    - BFS would usually be faster than a DFS.
- Advantages of either vary depending on the data and what you're looking for.

Reference: http://www.programmerinterview.com/index.php/data-structures/dfs-vs-bfs/

Adjacency Matrix

The drawbacks of using the adjacency matrix are:

- Increased memory as you need to declare N*N matrix where N is the total number of nodes.
- Redundancy of information, i.e. to represent an edge between A to B and B to A, it requires to set two Boolean flag in an adjacency matrix.

References:

- http://www.codeproject.com/KB/java/BFSDFS.aspx
- http://www.topcoder.com/tc?module=Static&d1=tutorials&d2=graphsDataStrucs2

Priority Queues

Input description - A set of records with numerically or otherwise totally ordered keys
Problem description - Build and maintain a data structure for quickly inserting and deleting records, while enabling quick access to the smallest or largest key in the set.
Discussion - Priority queues are useful data structures in simulations, particularly for maintaining a set of future events ordered by time so that we can quickly retrieve what the next thing to happen is. They are called ``priority'' queues because they enable you to retrieve items not by the insertion time (as in a stack or queue), nor by a key match (as in a dictionary), but by which item has the highest priority of retrieval.

Choice of data structures

- **Sorted array** or list - In a sorted array, it is very efficient to find and (by decrementing the top index) delete the smallest element. However, maintaining sortedness makes the insertion of new elements slow. Sorted arrays are suitable when there will be few insertions into the priority queue.
- **Binary heaps** - Heaps maintain an implicit binary tree structure in an array, such that the key of the root of any subtree is less than that of all its descendents. Thus the minimum key is always at the root of the heap. Binary heaps are the right answer whenever you know an upper bound on the number of items in your priority queue, since you must specify the array size at creation time.
- **Bounded height priority queue** - Bounded height priority queues are very useful in maintaining the vertices of a graph sorted by degree, which is a fundamental operation in graph algorithms.
- **Binary search trees** - Binary search trees make effective priority queues, since the smallest element is always the leftmost leaf, while the largest element is always the rightmost leaf. Binary tree heaps prove most appropriate when you also need to search a dictionary of the values, or if you have an unbounded key range and do not know the maximum priority queue size in advance.
- **Fibonacci and pairing heaps** - These complicated priority queues are designed to speed up decrease-key operations, where the priority of an item already in the priority queue is reduced. This arises, for example, in shortest path computations whenever we discover a shorter route to a vertex v than we had previously established. Thus v has a higher priority of being accessed next.

Graph Data Structures

Two options

a. Incidence matrices - which are analogous to adjacency matrices and requires n * m space, where m is number of hyperedges.
b. Bipartite incidence structures- which is analogous to adjacency list and hence suited for sparse hypergraphs.

Set Data Structures

Input description: A universe of items and a collection of subsets.
Problem description: Represent each subset so as to efficiently (1) test , (2) find the union or intersection of and , (3) insert or delete members of S.
Discussion: We distinguish sets from two other kinds of objects: strings and dictionaries. If there is no

fixed-size universal set, a collection of objects is best thought of as a dictionary.
Alternatives for representing arbitrary systems of subsets are:

- **Bit vectors** - If your universal set U contains n items, an n-bit vector or array can represent any subset. Bit i will be 1 if , otherwise bit i is 0. Since only one bit is used per element, bit vectors can be very space efficient for surprisingly large values of |U|. Element insertion and deletion simply flips the appropriate bit. Intersection and union are done by ``and-ing'' or ``or-ing'' the bits together. The only real drawback of a bit vector is that for sparse subsets, it takes O(n) time to explicitly identify all members of S.
- **Containers or dictionaries** - A subset can also be represented using a linked list, array, binary tree, or dictionary containing exactly the elements in the subset.
    - In many applications, the subsets are all pairwise disjoint, meaning that each element is in exactly one subset. For example, consider maintaining the connected components of a graph or the party affiliations of politicians. Each vertex/hack is in exactly one component/party. Such a system of subsets is called a set partition .
- **Dictionary with subset attribute** - If each item in a binary tree has associated a field recording the name of the subset it is in, set identification queries and single element modifications can be performed in the time it takes to search in the dictionary, typically . However, operations like performing the union of two subsets take time proportional to (at least) the sizes of the subsets, since each element must have its name changed. The need to perform such union operations quickly is the motivation for the ...
- **Union-Find Data Structure** - Suppose we represent a subset using a rooted tree, where each node points to its parent instead of its children. Further, let the name of the subset be the name of the item that is the root. Finding out which subset we are in is simple, for we keep traversing up the parent pointers until we hit the root. Unioning two subsets is also easy. Just make the root of one of two trees point to the other, so now all elements have the same root and thus the same subset name.

P and NP problem

- The P versus NP problem is a major unsolved problem in computer science. Informally, it asks whether every problem whose solution can be efficiently checked by a computer can also be efficiently solved by a computer.
- Suppose that solutions to a problem can be verified quickly. Then, can the solutions themselves also be computed quickly?

Polynomial time

- An algorithm is said to be of polynomial time if its running time is upper bounded by a polynomial expression in the size of the input for the algorithm, i.e., T(n) = O(nk) for some constant k.
- Problems for which a polynomial time algorithm exists belong to the complexity class P, which is central in the field of computational complexity theory.
- Polynomial time is a synonym for "tractable", "feasible", "efficient", or "fast". Some examples of polynomial time algorithms:
    - The quicksort sorting algorithm on n integers performs at most An2 operations for some constant A. Thus it runs in time O(n2) and is a polynomial time algorithm.
- All the basic arithmetic operations (addition, subtraction, multiplication, division, and comparison) can

be done in polynomial time.
- Maximum matchings in graphs can be found in polynomial time.

Complexity classes

The concept of polynomial time leads to several complexity classes in computational complexity theory. Some important classes defined using polynomial time are the following.

- **P:** The complexity class of decision problems that can be solved on a deterministic Turing machine in polynomial time.
- **NP:** The complexity class of decision problems that can be solved on a non-deterministic Turing machine in polynomial time.
- **ZPP:** The complexity class of decision problems that can be solved with zero error on a probabilistic Turing machine in polynomial time.
- **RP:** The complexity class of decision problems that can be solved with 1-sided error on a probabilistic Turing machine in polynomial time.
- **BPP:** The complexity class of decision problems that can be solved with 2-sided error on a probabilistic Turing machine in polynomial time.
- **BQP:** The complexity class of decision problems that can be solved with 2-sided error on a quantum Turing machine in polynomial time.
- **P** is the smallest time-complexity class on a deterministic machine which is robust in terms of machine model changes. (For example, a change from a single-tape Turing machine to a multi-tape machine can lead to a quadratic speedup, but any algorithm that runs in polynomial time under one model also does so on the other.) Any given abstract machine will have a complexity class corresponding to the problems which can be solved in polynomial time on that machine.

Applications

- Minimum Spanning Tree
- Shortest Path
- Traveling Salesman Problem
- Nearest Neighbor Search
- sparse matrix compression
- Algorithms from P to NP

Resources

- LEDA - Library of Efficient Data types and Algorithms
- Netlib - on-line repository of mathematical software
- The Stanford GraphBase
- Combinatorica - a collection of over 230 algorithms for combinatorics and graph theory written in Mathematica
- Algorithm Animations with XTango

- Animation for
    - Data structures - including AVL/red-black trees, and binary/Fibonacci heaps.
    - Sorting algorithms - including bubblesort, radixsort, quicksort, and shellsort.
    - Backtracking - including graph coloring, and both the eight-queens and knight's tour problems.
    - Geometric algorithms - including bin packing heuristics and the Graham scan/Jarvis march convex hull algorithms.
    - Graph algorithms - including minimum spanning trees and shortest paths.
    - String algorithms - including the Knuth-Morris-Pratt and Boyer-Moore algorithms

---

TBDs

- BST
- algorithm in search string

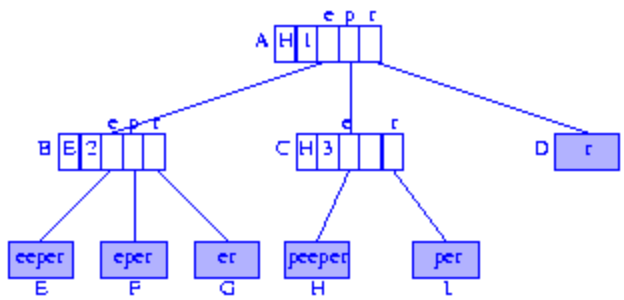
String matching algorithms

Index Method

Faster search algorithms are based on preprocessing of the text. After building a substring index, for example a suffix tree or suffix array, the occurrences of a pattern can be found quickly. As an example, a suffix tree can be built in T(n) time, and all z occurrences of a pattern can be found in O(m + z) time (if the alphabet size is viewed as a constant).

Suffix tree

- a suffix tree (also called PAT tree or, in an earlier form, position tree) is a data structure that presents the suffixes of a given string in a way that allows for a particularly fast implementation of many important string operations.
- The suffix tree for S is actually the compressed trie for the nonempty suffixes of the string S..
- Using Suffix Trees you can write something that will match your dictionary to your text in O(n+m+k) time where n is letters in your dictionary, m is letters in your text, and k is the number of matches



- If txt=t1t2...ti...tn is a string, then Ti=titi+1...tn is the suffix of txt that starts at position i, e.g.
  T1 = mississippi = txt
  T2 = ississippi
  T3 = ssissippi
  T4 = sissippi
  T5 = issippi
  T6 = ssippi
  T7 = sippi
  T8 = ippi
  T9 = ppi
  T10 = pi
  T11 = i
  T12 =

Suffix arrays

- A suffix array is a simple data structure that enables lookup of any substring of a text and identification of repeated substrings. It is more compact than a suffix tree and is amenable to storage in secondary memory.
- A suffix array is (notionally) a sorted list of the suffixes of a given string.
- A suffix is a substring that extends to the end of the given string.
- The sorted list is presented as an array of integers that identify the suffixes in order.
- The suffix array for the 11-letter text ABRACADABRA, with an endmark # that collates low, is given below, together with the corresponding suffixes.
  11 #
  10 A#
  7 ABRA#
  0 ABRACADABRA#
  3 ACADABRA#
  5 ADABRA#
  8 BRA#
  1 BRACADABRA#
  4 CADABRA#
  6 DABRA#
  9 RA#
  2 RACADABRA#

References:

- http://users.cis.fiu.edu/~weiss/dsaajava/code/DataStructures/